

Module 1 – Scalability

Code for modified warning_request.js:

```
// warning_request.js

// This code implements a TCP socket client that sends a request for weather conditions
// and listens for responses from the server.

const net = require("net");

const host = "127.0.0.1";
const port = 6000;

// Specify the area for which to request weather conditions
// List of areas can be defined based on the fire warning levels:
// Central, East Gippsland, Mallee, North Central, North East, Northern Country, South
// West, West and South Gippsland, Wimmera

const areas = [
  "Central",
  "East Gippsland",
  "Mallee",
  "North Central",
  "North East",
  "Northern Country",
  "South West",
  "West and South Gippsland",
  "Wimmera"
];
```

```
// const areaToRequest = "Central"; // Specify the area for which to request weather
conditions

// Create a TCP client that connects to the server
const client = net.createConnection(port, host, () => {
  console.log("Connected to Weather Service");

  // Set the encoding for the client
  setInterval(() => {
    const areaToRequest = areas[Math.floor(Math.random() * areas.length)]; // Randomly
select an area from the list

    client.write(` request,${areaToRequest}`);
    console.log(` Sent request for weather conditions in area: request,${areaToRequest}`);
  }, 2000); // Interval in milliseconds (2000ms = 2 seconds)
});

// Handle incoming data from the server
client.on("data", (data) => {
  console.log(` Received: ${data.toString()}`);
//   process.exit(0);
});

// Handle socket error event
client.on("error", (error) => {
  console.log(` Error: ${error.message}`);
});
```

```
// Handle socket end event

client.on("close", () => {

    console.log("Connection closed");

});
```

```
Request for weather conditions in area: North Central
Area: North Central, Temp: 4, Rain: 78, Wind: 39, Fire Warning Level: ok
rain : 185
wind : 89
temp : 25
Request for weather conditions in area: West and South Gippsland
Area: West and South Gippsland, Temp: 25, Rain: 185, Wind: 89, Fire Warning Level: ok
rain : 5
wind : 69
temp : 5
Request for weather conditions in area: North Central
Area: North Central, Temp: 5, Rain: 5, Wind: 69, Fire Warning Level: ok
rain : 147
wind : 91
temp : 2
Request for weather conditions in area: Central
Area: Central, Temp: 2, Rain: 147, Wind: 91, Fire Warning Level: ok
rain : 153
wind : 94
temp : 20
Request for weather conditions in area: Central
Area: Central, Temp: 20, Rain: 153, Wind: 94, Fire Warning Level: ok
rain : 129
wind : 60
temp : 35
Request for weather conditions in area: North Central
Area: North Central, Temp: 35, Rain: 129, Wind: 60, Fire Warning Level: ok
rain : 197
wind : 50
temp : 32
Request for weather conditions in area: Central
Area: Central, Temp: 32, Rain: 197, Wind: 50, Fire Warning Level: ok
rain : 84
wind : 68
temp : 13
Request for weather conditions in area: East Gippsland
Area: East Gippsland, Temp: 13, Rain: 84, Wind: 68, Fire Warning Level: ok
rain : 70
wind : 89
temp : 22
Request for weather conditions in area: North Central
Area: North Central, Temp: 22, Rain: 70, Wind: 89, Fire Warning Level: ok
rain : 80
wind : 47
```

Picture 1 – Results obtained from weather_service.js

```
PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 1 - Scalability\tech_task_week_2> node warning_request.js
Connected to Weather Service
Sent request for weather conditions in area: request,East Gippsland
Received: Area: East Gippsland, Temp: 5, Rain: 157, Wind: 62, Fire Warning Level: ok
Sent request for weather conditions in area: request,Northern Country
Received: Area: Northern Country, Temp: 9, Rain: 146, Wind: 31, Fire Warning Level: ok
Sent request for weather conditions in area: request,Mallee
Received: Area: Mallee, Temp: 6, Rain: 117, Wind: 19, Fire Warning Level: ok
Sent request for weather conditions in area: request,Northern Country
Received: Area: Northern Country, Temp: 24, Rain: 58, Wind: 65, Fire Warning Level: ok
Sent request for weather conditions in area: request,East Gippsland
Received: Area: East Gippsland, Temp: 33, Rain: 44, Wind: 39, Fire Warning Level: ok
Sent request for weather conditions in area: request,South West
Received: Area: South West, Temp: 14, Rain: 153, Wind: 57, Fire Warning Level: ok
Sent request for weather conditions in area: request,North East
Received: Area: North East, Temp: 4, Rain: 194, Wind: 65, Fire Warning Level: ok
Sent request for weather conditions in area: request,Mallee
Received: Area: Mallee, Temp: 21, Rain: 119, Wind: 40, Fire Warning Level: ok
Sent request for weather conditions in area: request,North Central
Received: Area: North Central, Temp: 4, Rain: 78, Wind: 39, Fire Warning Level: ok
Sent request for weather conditions in area: request,West and South Gippsland
Received: Area: West and South Gippsland, Temp: 25, Rain: 185, Wind: 89, Fire Warning Level: ok
Sent request for weather conditions in area: request,North Central
Received: Area: North Central, Temp: 5, Rain: 5, Wind: 69, Fire Warning Level: ok
```

Picture 2 – Results obtained from warning_request.js

Scalability Issues with the Current Architecture

The current implementation of the IoT weather warning service is built upon a simple client-server model, where all sensor nodes and user request nodes connect directly to a single `weather_service.js` server.

To simulate a more realistic environment for this task, the three weather-sensing nodes (wind, rain, and temperature) have been configured to generate a random value every 2 seconds, which is then sent to the server.

The `fire_node.js`, however, is set to an interval of 300,000ms (5 minutes) to avoid being blocked by the CFA website, from which it scrapes real fire warning data.

On the client side, the `warning_request.js` code has been modified to randomly select a location from a list of Victorian fire districts and send a request to the server every 2 seconds, effectively simulating hundreds of users concurrently accessing the service.

While this setup demonstrates the system's basic functionality, a key limitation is the reliance on randomly generated data for temperature, rainfall, and wind, as a real-time data source could not be identified for these parameters.

This contrasts with the fire data, which is obtained from a live source. This simple architecture, while functional for a limited scope, presents significant scalability challenges when the system needs to handle a larger number of devices and users.

On the other hand, these following can be expected from my current implementation:

- **Single Point of Failure (SPOF):**

- The most critical flaw of the current design is that the central server acts as a single point of failure.
- If the `weather_service.js` application or the physical machine it runs on fails, the entire system becomes inoperable.
- Both data collection from sensors and the ability to respond to user requests are completely halted.
- In a critical service like a weather warning system, this lack of resilience is unacceptable, as it could lead to missed warnings and potential safety risks.

- **Resource Bottleneck:**

- As the number of sensor nodes and user requests increases, the single server becomes a resource bottleneck.
- It must manage a growing number of simultaneous TCP connections and process all incoming data and outgoing responses.

- This leads to a strain on the server's CPU, memory, and network resources.
 - As a result, the system's performance will degrade, latency will increase, and at a certain point, the server will become overwhelmed and unable to respond reliably.
 - This limits the system's ability to scale horizontally, as all processing is concentrated on a single machine.
- **Lack of Data Persistence:**
 - The current system stores all sensor data in volatile in-memory variables.
 - If the server were to crash or be restarted, all collected weather data would be lost. This prevents any form of historical data analysis, which is crucial for identifying long-term trends, improving warning accuracy, or providing post-event analysis.

Proposed Architectural Improvements for Scalability

To address the issues and create a more robust and scalable system, a shift from the current centralized model to a distributed architecture is required. The following improvements are proposed to enhance the system's resilience, performance, and scalability.

- **Decoupling with a Message Queue:**
 - To eliminate the single point of failure and reduce the load on the central server, a **Message Queue** (e.g., RabbitMQ, Apache Kafka) should be introduced. In this new model, sensor nodes would become **producers**, publishing their data to the message queue.
 - The weather_service.js application would become a **consumer**, subscribing to the data streams from the queue. This approach decouples the producers from the consumers.
 - If the weather_service.js is temporarily unavailable, sensor data can still be queued and processed once the service is restored, ensuring no data is lost. This also offloads the burden of managing numerous connections from the server to the message broker.
- **Integrating a Database for Data Persistence:**
 - To ensure data durability and enable historical analysis, a dedicated database should be integrated into the architecture.
 - The weather_service.js application would be modified to write all incoming sensor data to the database instead of storing it in memory.

- This ensures that even in the event of a system failure, all collected data is preserved.
- A time-series database (e.g., InfluxDB) would be particularly well-suited for this type of time-stamped sensor data.
- **Implementing Horizontal Scaling with a Load Balancer:**
 - To overcome the resource bottleneck and further increase fault tolerance, the `weather_service.js` should be horizontally scaled.
 - This involves running multiple instances of the server behind a **Load Balancer**.
 - The load balancer would intelligently distribute incoming requests from user nodes across the available server instances.
 - This not only increases the system's capacity to handle a high volume of concurrent requests but also provides redundancy.
 - If one of the server instances fails, the load balancer will simply route traffic to the remaining healthy servers, providing a seamless and uninterrupted service to users.

Module 2 – Scalable IoT Applications

Screenshots

```
JS heat_sensor.js X
iot_sensors > JS heat_sensor.js > ...
1 // heat_sensor.js
2 // This module defines a HeatSensor class that simulates a heat sensor device.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/forest/sensors/heat';
7
8 //const heat_sensor_ids = ['heat_sensor_1', 'heat_sensor_2', 'heat_sensor_3'];
9 //const heat_topics = heat_sensor_ids.map(id => `/forest/sensors/heat/${id}`);
10
11 client.on('connect', () => {
12
13     console.log('mqtt connected');
14
15     setInterval(function() {
16
17         //heat_topics.forEach(topic => {
18
19             const heatLevel = Math.floor(Math.random() * 101); // Random heat level between 0 and 100
20
21             const message = JSON.stringify({ level: heatLevel });
22
23             client.publish(topic, message);
24
25             console.log(`Published to Topic ${topic} with Message: ${message}`);
26
27         //});
28     }, 2000);
29 });
```

Picture 3 – Code of heat_sensor.js

```
JS humidity_sensor.js X
iot_sensors > JS humidity_sensor.js > ...
You, 57 seconds ago | 1 author (You)
1 // humidity_sensor.js
2 // This module defines a WindSensor class that simulates a wind sensor device.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/forest/sensors/humidity';
7
8 //const smoke_sensor_ids = ['smoke_sensor_1', 'smoke_sensor_2', 'smoke_sensor_3'];
9 //const smoke_topics = smoke_sensor_ids.map(id => `/forest/sensors/smoke/${id}`);
10
11 client.on('connect', () => {
12
13     console.log('mqtt connected');
14
15     setInterval(function() {
16
17         //smoke_topics.forEach(topic => {
18
19             const humidityLevel = Math.floor(Math.random() * 101); // Random smoke level between 0 and 100
20
21             const message = JSON.stringify({ level: humidityLevel });
22
23             client.publish(topic, message);
24             console.log(`Published to Topic ${topic} with Message: ${message}`);
25
26         //});
27     }, 2000);
28 });
You, 56 seconds ago • commit module 2 - C ...
```

Picture 4 – Code of humidity_sensor.js

```
JS smoke_sensor.js X
iot_sensors > JS smoke_sensor.js > ...
You, 2 minutes ago | 1 author (You)
1 // smoke_sensor.js
2 // This module defines a SmokeSensor class that simulates a smoke sensor device.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/forest/sensors/smoke';
7
8 //const smoke_sensor_ids = ['smoke_sensor_1', 'smoke_sensor_2', 'smoke_sensor_3'];
9 //const smoke_topics = smoke_sensor_ids.map(id => `/forest/sensors/smoke/${id}`);
10
11 client.on('connect', () => {
12
13     console.log('mqtt connected');
14
15     setInterval(function() {
16
17         //smoke_topics.forEach(topic => {
18
19             const smokeLevel = Math.floor(Math.random() * 101); // Random smoke level between 0 and 100
20
21             const message = JSON.stringify({ level: smokeLevel });
22
23             client.publish(topic, message);
24             console.log(`Published to Topic ${topic} with Message: ${message}`);
25
26         //});
27     }, 2000);
28 });
```

Picture 5 – Code of smoke_sensor.js

```
JS wind_sensor.js X
iot_sensors > JS wind_sensor.js > ...
You, 2 minutes ago | 1 author (You)
1 // wind_sensor.js
2 // This module defines a WindSensor class that simulates a wind sensor device.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/forest/sensors/wind';
7
8 //const smoke_sensor_ids = ['smoke_sensor_1', 'smoke_sensor_2', 'smoke_sensor_3'];
9 //const smoke_topics = smoke_sensor_ids.map(id => `/forest/sensors/smoke/${id}`);
10
11 client.on('connect', () => {
12
13     console.log('mqtt connected');
14
15     setInterval(function() {
16
17         //smoke_topics.forEach(topic => {
18
19             const windLevel = Math.floor(Math.random() * 101); // Random smoke level between 0 and 100
20
21             const message = JSON.stringify({ level: windLevel });
22
23             client.publish(topic, message);
24             console.log(`Published to Topic ${topic} with Message: ${message}`);
25
26         //});
27     }, 2000);
28 });
```

You, 2 minutes ago • commit module 2 - C

Picture 6 – Code of wind_sensor.js

```
JS fire_departments.js X
clients > JS fire_departments.js > client.on('connect') callback
You, 3 minutes ago | 1 author (You)
1 // fire_departments.js
2 // This module defines a FireDepartments class that simulates a fire department client.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/alert/fire_departments';
7
8 client.on('connect', () => {
9   client.subscribe(topic);
10  console.log('Fire Department Client connected and subscribed to topic:', topic);
11 });
12
13 client.on('message', (topic, message) => {
14   console.log(`Fire Department Client received message on topic ${topic}: ${message.toString()}`);
15 });
```

Picture 7 – Code of fire_departments.js

```
JS local_homeowners.js X
clients > JS local_homeowners.js > ...
You, 3 minutes ago | 1 author (You)
1 // local_homeowners.js
2 // This module defines a LocalHomeowners class that simulates a local homeowner client.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/alert/local_homeowners';
7
8 client.on('connect', () => {
9   client.subscribe(topic);
10  console.log('Local Homeowners Client connected and subscribed to topic:', topic);
11 });
12
13 client.on('message', (topic, message) => {
14   console.log(`Local Homeowners Client received message on topic ${topic}: ${message.toString()}`);
15 });
```

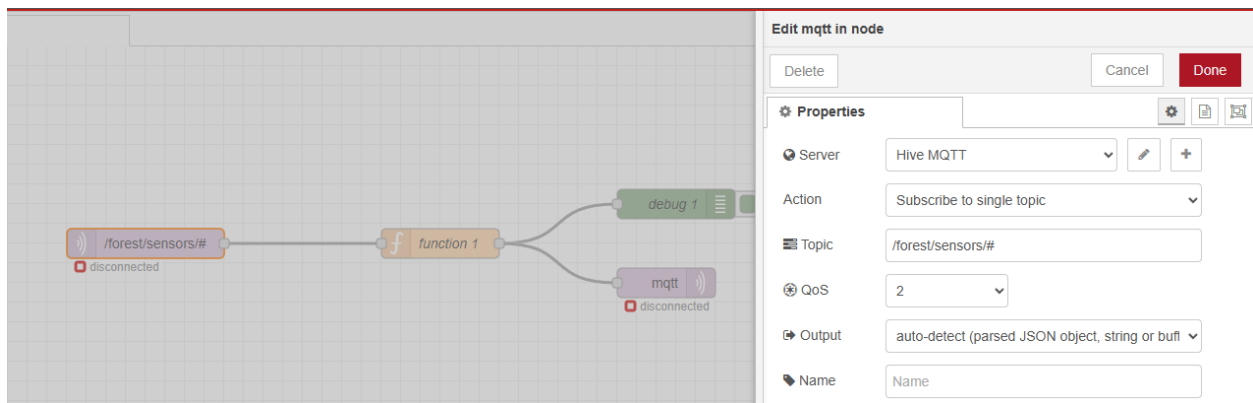
Picture 8 – Code of the local_homeowners.js

```
JS news_organizations.js X
clients > JS news_organizations.js > ...
You, 4 minutes ago | 1 author (You)
1 // news_organizations.js
2 // This module defines a NewsOrganizations class that simulates a news organization client.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/alert/news_organizations';
7
8 client.on('connect', () => {
9   client.subscribe(topic);
10  console.log('News Organization Client connected and subscribed to topic:', topic);
11 });
12
13 client.on('message', (topic, message) => {
14   console.log(`News Organization Client received message on topic ${topic}: ${message.toString()}`);
15 });
```

Picture 9 – Code of the news_organizations.js

```
JS social_media.js X
clients > JS social_media.js > ...
You, 4 minutes ago | 1 author (You)
1 // social_media.js
2 // This module defines a SocialMedia class that simulates a social media client.
3 const mqtt = require('mqtt');
4 const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6 const topic = '/alert/social_media';
7
8 client.on('connect', () => {
9   client.subscribe(topic);
10  console.log('Social Media Client connected and subscribed to topic:', topic);
11 });
12
13 client.on('message', (topic, message) => {
14   console.log(`Social Media Client received message on topic ${topic}: ${message.toString()}`);
15 });
```

Picture 10 – Code from social_media.js



Picture 11 – MQTT Input Node on Node-RED.

Code for “function” node on Node-RED

```
// Use flow concept to store variables send from IoT Sensors
// Create an empty object if 'alertStates' has not existed before.
var flowContext = flow.get('alertStates') || {};

// Split the incoming msg into parts for further logic implementations
var parts = msg.topic.split('/');
var sensorType = parts[3];
var alertFactor = 0;
var sensorLevel = msg.payload.level;

// Store the level of the current sensor in flow context
```

```

// This allows you to access the latest value from any sensor
flowContext[sensorType] = { level: sensorLevel, alert: 0 };

if (sensorType === "smoke") {
    if (sensorLevel >= 50) {
        flowContext[sensorType].alert = 1;
    }
} else if (sensorType === "heat") {
    if (sensorLevel >= 60) {
        flowContext[sensorType].alert = 1;
    }
} else if (sensorType === "wind") {
    if (sensorLevel >= 70) {
        flowContext[sensorType].alert = 1;
    }
} else if (sensorType === "humidity") {
    if (sensorLevel <= 20) {
        flowContext[sensorType].alert = 1;
    }
}

// Save the updated flow context
flow.set('alertStates', flowContext);

// Calculate the total number of alerts
var totalAlerts = 0;
for (var key in flowContext) {
    totalAlerts += flowContext[key].alert;
}

// Define an array to store warning messages
var alertsToSend = [];

// Main Alert Condition (>= 2 factors)
if (totalAlerts >= 2) {
    alertsToSend.push({
        topic: "/alert/local_homeowners",
        payload: `Forest Fire is happening - Please evacuate immediately!!!`
    });

    alertsToSend.push({
        topic: "/alert/news_organizations",
        payload: `Forest Fire is happening - Please report this LIVE to inform local homeowners to be cautious!!!`
    });
}

```

```
    alertsToSend.push({
      topic: "/alert/social_media",
      payload: `Forest Fire is happening - Please stay away from it at the
moment!!!`
    });
  }

  // Specific Alert for Fire Departments
  var currentSmokeLevel = flowContext['smoke'] ? flowContext['smoke'].level : 0;
  var currentHeatLevel = flowContext['heat'] ? flowContext['heat'].level : 0;

  if (currentSmokeLevel >= 50 && currentHeatLevel >= 80) {
    alertsToSend.push({
      topic: "/alert/fire_departments",
      payload: `Forest Fire is happening - Please send men to fight it!!! Smoke
Level is ${currentSmokeLevel}, Heat Level is ${currentHeatLevel}`
    });
  }

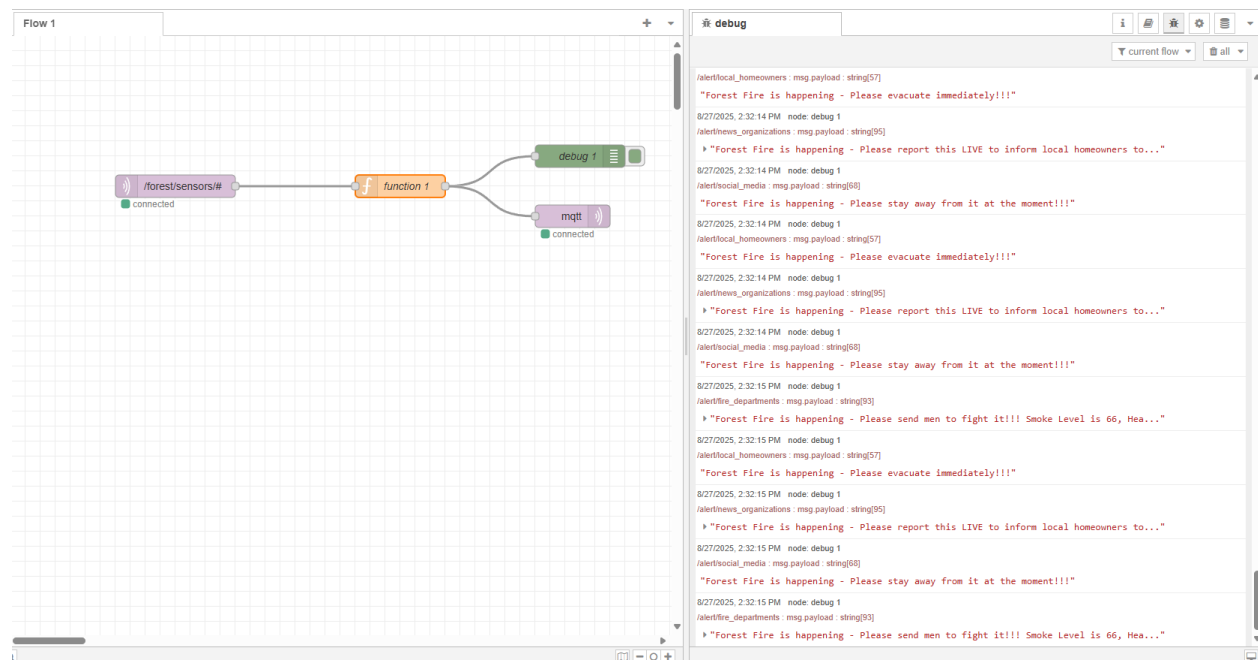
  // Send alerts if any are present
  if (alertsToSend.length > 0) {
    return [alertsToSend];
  }

  return null;
}
```

Results:

```
[4] Published to Topic /forest/sensors/heat with Message: {"level":88}
[7] Published to Topic /forest/sensors/wind with Message: {"level":93}
[5] Published to Topic /forest/sensors/humidity with Message: {"level":42}
[6] Published to Topic /forest/sensors/smoke with Message: {"level":34}
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
owners to be cautious!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
owners to be cautious!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
owners to be cautious!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[4] Published to Topic /forest/sensors/heat with Message: {"level":46}
[7] Published to Topic /forest/sensors/wind with Message: {"level":3}
[5] Published to Topic /forest/sensors/humidity with Message: {"level":0}
[6] Published to Topic /forest/sensors/smoke with Message: {"level":47}
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
owners to be cautious!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[7] Published to Topic /forest/sensors/wind with Message: {"level":81}
[4] Published to Topic /forest/sensors/heat with Message: {"level":77}
[6] Published to Topic /forest/sensors/smoke with Message: {"level":5}
[5] Published to Topic /forest/sensors/humidity with Message: {"level":10}
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
owners to be cautious!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
owners to be cautious!!!
```

Picture 12 – Results obtained from the client-side (all clients: fire_departments.js, local_homeowners.js, news_organizations.js, social_media.js).



Picture 13 – Results yielded from “debug” node in Red-NODE.

Explanations:

This project, a prototype of a forest fire monitoring and alarm system, was built using Node.js, MQTT, and Node-RED.

The core objective was to demonstrate a scalable and intelligent IoT solution capable of processing data from multiple sensors and delivering tailored alerts to different stakeholder groups, namely local homeowners, news organizations, social media, and fire departments.

1. Solution Overview

The solution is structured as a robust, event-driven system:

- **Sensors:** Simulated using Node.js scripts, these act as data publishers, sending mock sensor readings (e.g., smoke, heat, wind, humidity levels) to an MQTT broker.
- **Node-RED:** This acts as the central processing unit, subscribing to the raw sensor data, applying complex business logic, and making decisions on which alerts to send.
- **Clients:** Simulated as separate Node.js scripts, these act as subscribers, listening to specific alerts on dedicated MQTT topics.

2. Key Components and Their Functionality

a) MQTT Client Scripts (Sensors & Clients)

We implemented two main types of Node.js scripts. Both connect to the public MQTT broker `broker.hivemq.com` and operate asynchronously.

- **Sensor Scripts:** Scripts like `smoke_sensor.js` and `heat_sensor.js` continuously publish messages to specific, hierarchical topics (e.g., `/forest/sensors/smoke` and `/forest/sensors/heat`). The hierarchical topic structure allows the system to easily differentiate between sensor types.
- **Client Scripts:** Each client file (e.g., `local_homeowners.js`, `fire_departments.js`) subscribes to a unique, pre-defined alert topic. This ensures that each client only receives messages relevant to their role.

b) Node-RED Flow and Business Logic

The core intelligence of the system resides in the Node-RED flow.

- **MQTT In Node:** This node is configured to subscribe to the broad topic `/forest/sensors/#`, allowing it to receive data from all sensor types (smoke, heat, wind, humidity) with a single node.
- **Function Node:** This is the brain of the flow. It processes the incoming messages and applies our custom logic.
 - **Flow Context:** To implement the rule that an alert requires multiple factors, we utilized Node-RED's flow context. The `flow.get('alertStates')` variable acts as a temporary memory, storing the current alert status and sensor level for each sensor type.
 - **Complex Logic:** The code checks the sensor type (`msg.topic.split('/')`) and updates the `alertStates` dictionary.
 - **Conditional Alerts:** The script checks two primary conditions:
 1. **General Alert:** If the total number of alerting sensors (`totalAlerts >= 2`), a general alert is pushed to topics for homeowners, news, and social media.
 2. **Fire Department Alert:** If specific, critical levels are detected (e.g., `smokeLevel >= 50 && heatLevel >= 80`), a more detailed alert is pushed to the fire department's topic.
 - **Returning Multiple Messages:** The function returns an array of messages (`return [alertsToSend];`), which Node-RED automatically handles by sending each message to its respective topic.
- **MQTT Out Nodes:** Two separate mqtt out nodes are used. They receive the messages from the function node and publish them to their assigned topics (e.g., `/alert/local_homeowners`, `/alert/fire_departments`), completing the data flow.

c) Automation

To streamline testing and deployment, we used concurrently within the `package.json` file. This allows all sensor and client scripts to be started simultaneously with a single command (`npm start`), eliminating the need to manage multiple terminal windows.

